

```

import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertFalse;
import static org.junit.Assert.assertTrue;

import org.junit.Test;

import components.map.Map;

/**
 * JUnit test fixture for {@code Map<String, String>}s constructor and kernel
 * methods.
 *
 * @author cl_c2231am03 (Tyler zhuangzhuang He)
 */
public abstract class MapTest {

    /**
     * Invokes the appropriate {@code Map} constructor for the implementation
     * under test and returns the result.
     *
     * @return the new map
     * @ensures constructorTest = {}
     */
    protected abstract Map<String, String> constructorTest();

    /**
     * Invokes the appropriate {@code Map} constructor for the reference
     * implementation and returns the result.
     *
     * @return the new map
     * @ensures constructorRef = {}
     */
    protected abstract Map<String, String> constructorRef();

    /**
     *
     * Creates and returns a {@code Map<String, String>} of the implementation
     * under test type with the given entries.
     *
     * @param args
     *         the (key, value) pairs for the map
     * @return the constructed map
     * @requires <pre>
     * [args.length is even] and
     * [the 'key' entries in args are unique]
     * </pre>
     * @ensures createFromArgsTest = [pairs in args]
     */
    private Map<String, String> createFromArgsTest(String... args) {
        assert args.length % 2 == 0 : "Violation of: args.length is even";
        Map<String, String> map = this.constructorTest();
        for (int i = 0; i < args.length; i += 2) {
            assert !map.containsKey(args[i]) : ""
                + "Violation of: the 'key' entries in args are unique";
            map.add(args[i], args[i + 1]);
        }
        return map;
    }

    /**
     *
     * Creates and returns a {@code Map<String, String>} of the reference
     * implementation type with the given entries.
     *
     * @param args
     *         the (key, value) pairs for the map
     * @return the constructed map
     * @requires <pre>

```

```

    * [args.length is even] and
    * [the 'key' entries in args are unique]
    * </pre>
    * @ensures createFromArgsRef = [pairs in args]
    */
private Map<String, String> createFromArgsRef(String... args) {
    assert args.length % 2 == 0 : "Violation of: args.length is even";
    Map<String, String> map = this.constructorRef();
    for (int i = 0; i < args.length; i += 2) {
        assert !map.containsKey(args[i]) : ""
            + "Violation of: the 'key' entries in args are unique";
        map.add(args[i], args[i + 1]);
    }
    return map;
}

// test cases for constructor
@Test
public final void testConstructor() {
    Map<String, String> map = this.constructorTest();
    Map<String, String> expected = this.constructorRef();

    assertEquals(expected, map);
}

// test cases for add: add to empty and non-empty maps
@Test
public final void testAddToEmpty() {
    Map<String, String> map = this.constructorTest();
    Map<String, String> expected = this.createFromArgsRef("red", "apple");

    map.add("red", "apple");

    assertEquals(expected, map);
}

@Test
public final void testAddToNonEmpty() {
    Map<String, String> map = this.createFromArgsTest("red", "apple",
        "yellow", "banana");
    Map<String, String> expected = this.createFromArgsRef("red", "apple",
        "yellow", "banana", "green", "lime");

    map.add("green", "lime");

    assertEquals(expected, map);
}

// test cases for remove: only entry, first entry, middle entry, last entry
@Test
public final void testRemoveOnlyEntry() {
    Map<String, String> map = this.createFromArgsTest("red", "apple");
    Map<String, String> expected = this.constructorRef();

    Map.Pair<String, String> removed = map.remove("red");

    assertEquals("red", removed.key());
    assertEquals("apple", removed.value());
    assertEquals(expected, map);
}

@Test
public final void testRemoveFirstEntry() {
    Map<String, String> map = this.createFromArgsTest("red", "apple",
        "yellow", "banana", "green", "lime");
    Map<String, String> expected = this.createFromArgsRef("yellow",
        "banana", "green", "lime");

    Map.Pair<String, String> removed = map.remove("red");

```

```
    assertEquals("red", removed.key());
    assertEquals("apple", removed.value());
    assertEquals(expected, map);
}

@Test
public final void testRemoveMiddleEntry() {
    Map<String, String> map = this.createFromArgsTest("red", "apple",
        "yellow", "banana", "green", "lime");
    Map<String, String> expected = this.createFromArgsRef("red", "apple",
        "green", "lime");

    Map.Pair<String, String> removed = map.remove("yellow");

    assertEquals("yellow", removed.key());
    assertEquals("banana", removed.value());
    assertEquals(expected, map);
}

@Test
public final void testRemoveLastEntry() {
    Map<String, String> map = this.createFromArgsTest("red", "apple",
        "yellow", "banana", "green", "lime");
    Map<String, String> expected = this.createFromArgsRef("red", "apple",
        "yellow", "banana");

    Map.Pair<String, String> removed = map.remove("green");

    assertEquals("green", removed.key());
    assertEquals("lime", removed.value());
    assertEquals(expected, map);
}

// test cases for removeAny
@Test
public final void testRemoveAnyOneEntry() {
    Map<String, String> map = this.createFromArgsTest("red", "apple");
    Map<String, String> expected = this.constructorRef();

    Map.Pair<String, String> removed = map.removeAny();

    assertEquals("red", removed.key());
    assertEquals("apple", removed.value());
    assertEquals(expected, map);
}

@Test
public final void testRemoveAnyNonEmpty() {
    Map<String, String> map = this.createFromArgsTest("red", "apple",
        "yellow", "banana", "green", "lime");

    Map.Pair<String, String> removed = map.removeAny();

    assertFalse(map.containsKey(removed.key()));
    assertEquals(2, map.size());
}

// test cases for value: only entry, first entry, middle entry, last entry
@Test
public final void testValueOnlyEntry() {
    Map<String, String> map = this.createFromArgsTest("red", "apple");
    Map<String, String> expected = this.createFromArgsRef("red", "apple");

    String value = map.value("red");

    assertEquals("apple", value);
    assertEquals(expected, map);
}
```

```
@Test
public final void testValueFirstEntry() {
    Map<String, String> map = this.createFromArgsTest("red", "apple",
        "yellow", "banana", "green", "lime");
    Map<String, String> expected = this.createFromArgsRef("red", "apple",
        "yellow", "banana", "green", "lime");

    String value = map.value("red");

    assertEquals("apple", value);
    assertEquals(expected, map);
}

@Test
public final void testValueMiddleEntry() {
    Map<String, String> map = this.createFromArgsTest("red", "apple",
        "yellow", "banana", "green", "lime");
    Map<String, String> expected = this.createFromArgsRef("red", "apple",
        "yellow", "banana", "green", "lime");

    String value = map.value("yellow");

    assertEquals("banana", value);
    assertEquals(expected, map);
}

@Test
public final void testValueLastEntry() {
    Map<String, String> map = this.createFromArgsTest("red", "apple",
        "yellow", "banana", "green", "lime");
    Map<String, String> expected = this.createFromArgsRef("red", "apple",
        "yellow", "banana", "green", "lime");

    String value = map.value("green");

    assertEquals("lime", value);
    assertEquals(expected, map);
}

// test cases for hasKey: empty, present, absent
@Test
public final void testHasKeyEmptyFalse() {
    Map<String, String> map = this.constructorTest();
    Map<String, String> expected = this.constructorRef();

    boolean result = map.hasKey("red");

    assertFalse(result);
    assertEquals(expected, map);
}

@Test
public final void testHasKeyPresentTrue() {
    Map<String, String> map = this.createFromArgsTest("red", "apple",
        "yellow", "banana");
    Map<String, String> expected = this.createFromArgsRef("red", "apple",
        "yellow", "banana");

    boolean result = map.hasKey("yellow");

    assertTrue(result);
    assertEquals(expected, map);
}

@Test
public final void testHasKeyAbsentFalse() {
    Map<String, String> map = this.createFromArgsTest("red", "apple",
        "yellow", "banana");
```

```
        Map<String, String> expected = this.createFromArgsRef("red", "apple",
            "yellow", "banana");

        boolean result = map.containsKey("green");

        assertFalse(result);
        assertEquals(expected, map);
    }

    // test cases for size: empty and non-empty maps
    @Test
    public final void testSizeEmpty() {
        Map<String, String> map = this.constructorTest();
        Map<String, String> expected = this.constructorRef();

        int size = map.size();

        assertEquals(0, size);
        assertEquals(expected, map);
    }

    @Test
    public final void testSizeNonEmpty() {
        Map<String, String> map = this.createFromArgsTest("red", "apple",
            "yellow", "banana", "green", "lime");
        Map<String, String> expected = this.createFromArgsRef("red", "apple",
            "yellow", "banana", "green", "lime");

        int size = map.size();

        assertEquals(3, size);
        assertEquals(expected, map);
    }
}
```